Internship Report

(Day 4)

Report Type	Internship Daily Report
Day	Day 4
Name	Ibrahim Hasan
Date	20th May 2026
Task	Build a small Employee Management System Database with 4 tables. Then run SQL queries and document results.

Summary of Work

Step 1 — Create Database & Tables

The `employee_management` database was created and activated using the `CREATE DATABASE` and `USE` commands via the MySQL Shell and phpMyAdmin SQL interface. Four tables were then defined – `departments`, `employees`, `projects`, and `employee_projects` – each structured with appropriate data types, primary keys, and foreign key references to enforce relational integrity.

```sql
CREATE DATABASE employee_management;
USE employee_management;
```

Figure 1: CREATE DATABASE and USE statements executed in MYSQL Shell.

departments table:

Column	Type	Constraint	Null	Notes
`department_id`	`INT`	PK, AUTO_INCREMENT	No	Auto PK
`department_name`	`VARCHAR(100)`	NOT NULL	No	Required
`location`	`VARCHAR(100)`	–	Yes	Optional

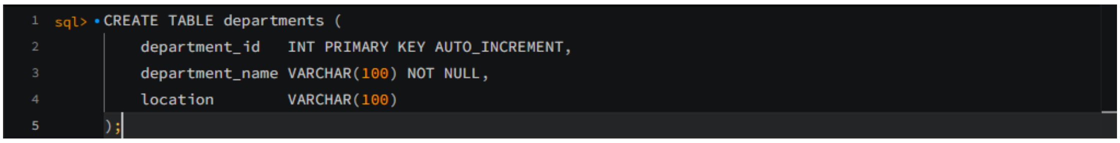

Figure 2: CREATE TABLE departments in MYSQL Shell.

employees table:

Column	Type	Constraint	Null	Notes
`employee_id`	`INT`	PK, AUTO_INCREMENT	No	Auto PK
`employee_name`	`VARCHAR(100)`	NOT NULL	No	Required
`email`	`VARCHAR(100)`	UNIQUE	Yes	Unique
`phone`	`VARCHAR(20)`	–	Yes	Optional
`salary`	`DECIMAL(10,2)`	NOT NULL	No	Required
`joining_date`	`DATE`	NOT NULL	No	Required
`department_id`	`INT`	FK → departments	Yes	FK ref
`status`	`TINYINT(1)`	DEFAULT 1	Yes	1=Active

```sql
sql> •CREATE TABLE employees (
    employee_id   INT PRIMARY KEY AUTO_INCREMENT,
    employee_name VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE,
    phone         VARCHAR(20),
    salary        DECIMAL(10,2) NOT NULL,
    joining_date  DATE NOT NULL,
    department_id INT,
    status        BOOLEAN DEFAULT 1,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
```

Figure 3: CREATE TABLE employees with SQL Shell.

projects table:

Column	Type	Constraint	Null	Notes
`project_id`	`INT`	PK, AUTO_INCREMENT	No	Auto PK
`project_name`	`VARCHAR(100)`	NOT NULL	No	Required
`start_date`	`DATE`	–	Yes	Optional
`end_date`	`DATE`	–	Yes	Optional
`budget`	`DECIMAL(12,2)`	–	Yes	Optional

```sql
sql> •CREATE TABLE projects (
    project_id   INT PRIMARY KEY AUTO_INCREMENT,
    project_name VARCHAR(100) NOT NULL,
    start_date   DATE,
    end_date     DATE,
    budget       DECIMAL(12,2)
);
```

Figure 4: CREATE TABLE projects with MYSQL Shell.

employee_projects table:

Column	Type	Constraint	Null	Notes
`id`	`INT`	PK, AUTO_INCREMENT	No	Auto PK
`employee_id`	`INT`	FK → employees	Yes	FK ref
`project_id`	`INT`	FK → projects	Yes	FK ref
`assigned_date`	`DATE`	–	Yes	Optional

```sql
sql> CREATE TABLE employee_projects (
    id            INT PRIMARY KEY AUTO_INCREMENT,
    employee_id   INT,
    project_id    INT,
    assigned_date DATE,
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),
    FOREIGN KEY (project_id)  REFERENCES projects(project_id)
);
```

Figure 5: CREATE TABLE employee projects with MYSQL Shell.

Step 2 — Insert Sample Data

Bulk `INSERT INTO` statements were executed to populate each table with 25 rows of realistic sample data. The `departments` table was seeded with Bangladeshi city locations; the `employees` table with local names, unique emails, and salary figures; the `projects` table with real-world project titles and budgets; and the `employee_projects` junction table with assignment records linking employees to projects.

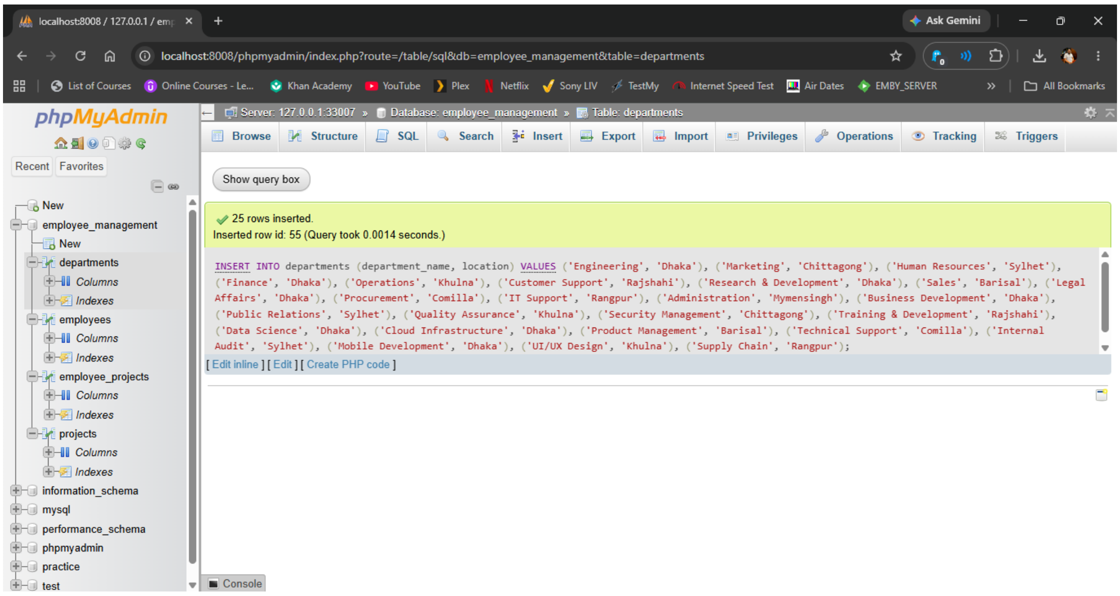

Figure 6: 25 rows inserted into ‘departments‘ on phpMyAdmin confirmation

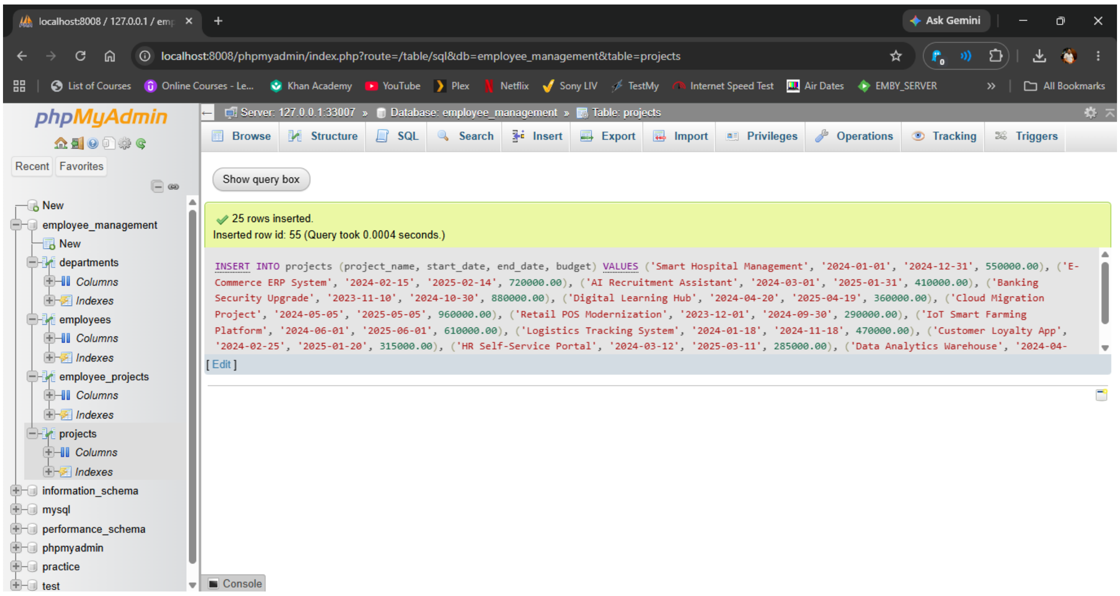

Figure 7: 25 rows inserted into ‘projects‘ on phpMyAdmin confirmation.

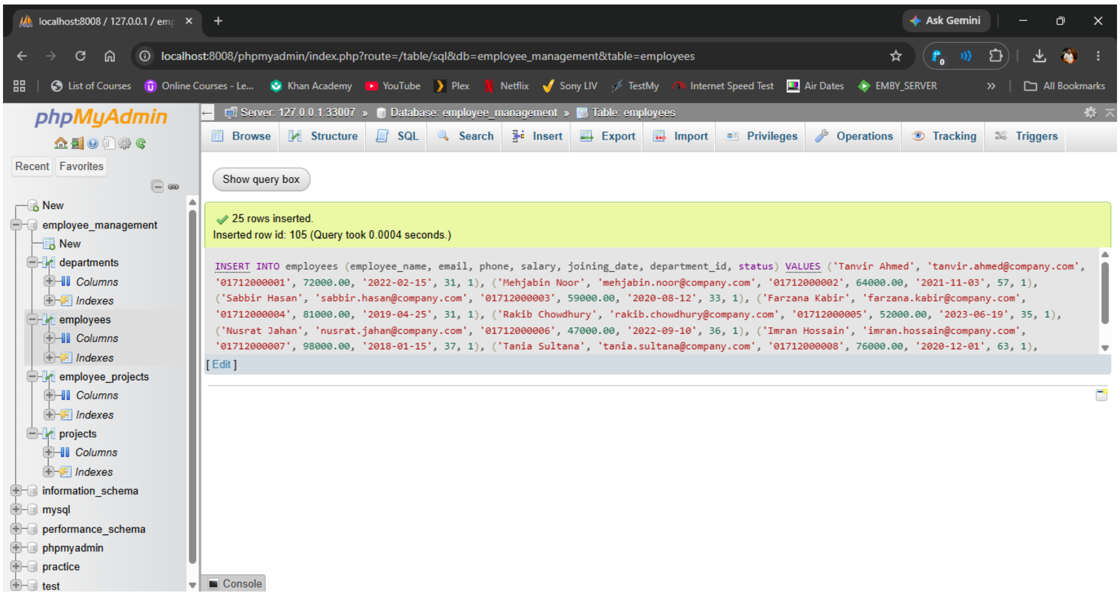

Figure 8: 25 rows inserted into ‘employees‘ on phpMyAdmin confirmation.

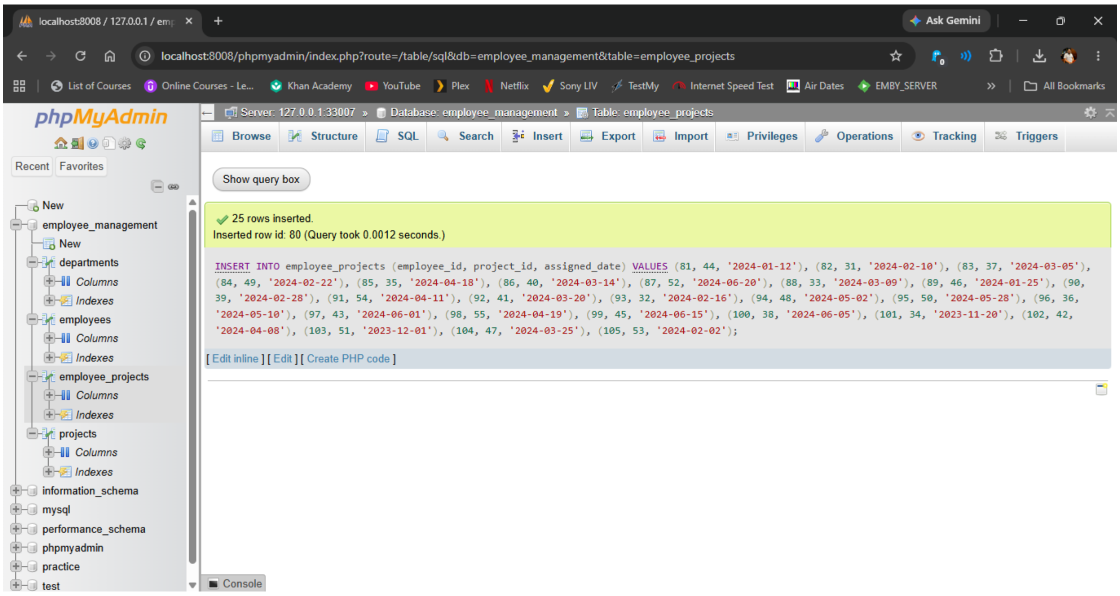

Figure 9: 25 rows inserted into 'employee projects' on phpMyAdmin confirmation.

Step 3 — Run SQL Queries

A series of SQL queries were executed against the populated database to retrieve and analyse data across multiple tables. Each query is documented below with its purpose and result.

Query 1 — Show all employees

```
SELECT * FROM employees;
```

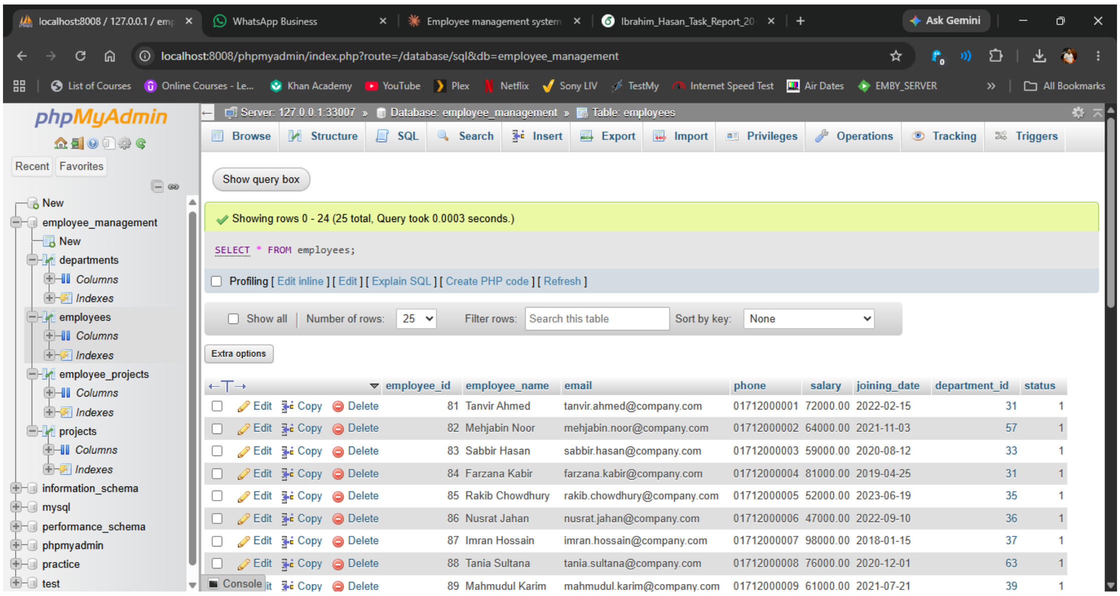

Figure 10: Query 1 result output in phpMyAdmin..

Query 2 — Show employees with salary above 60000

```sql
SELECT employee_name , salary FROM employees WHERE salary >
    60000;
```

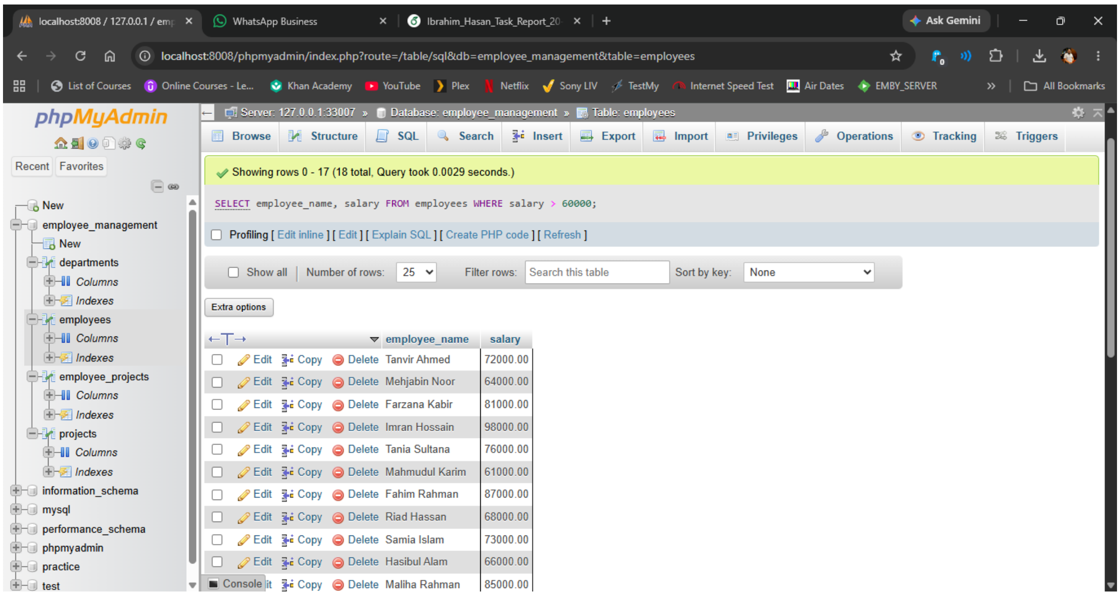

Figure 11: Query 2 result output in phpMyAdmin.

Query 3 — Show employees ordered by salary (highest first)

```sql
SELECT employee_name , salary FROM employees ORDER BY salary DESC
    ;
```

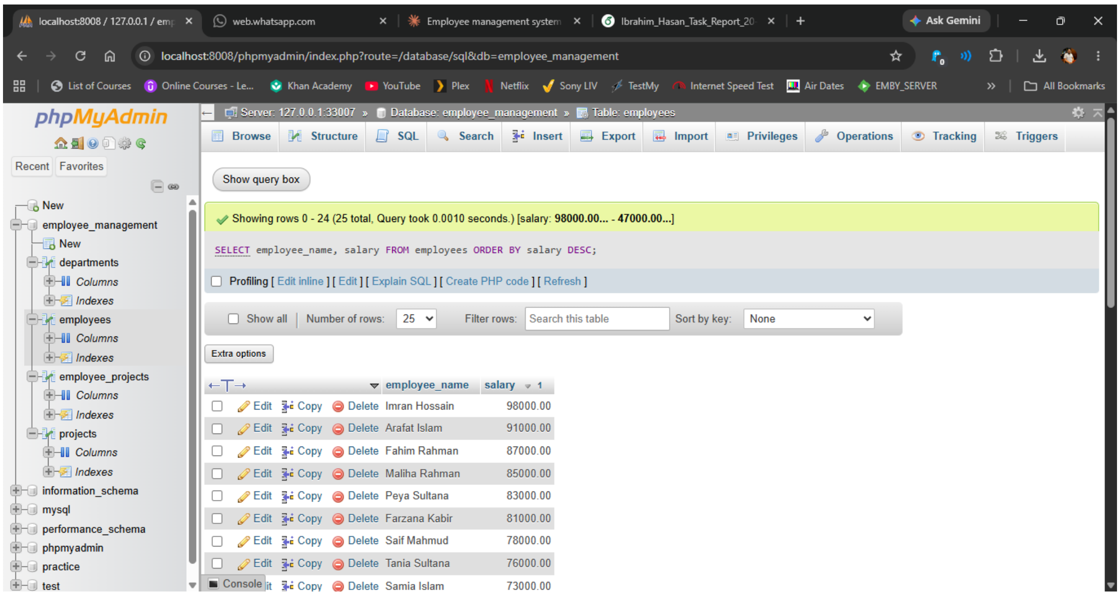

Figure 12: Query 3 result output in phpMyAdmin.

Step 4 — Verify Results & Document

All query outputs were reviewed to confirm accuracy and consistency with the inserted data. The database structure was verified through phpMyAdmin's Relation View, confirming that all foreign key constraints between the four tables are correctly established and operational.

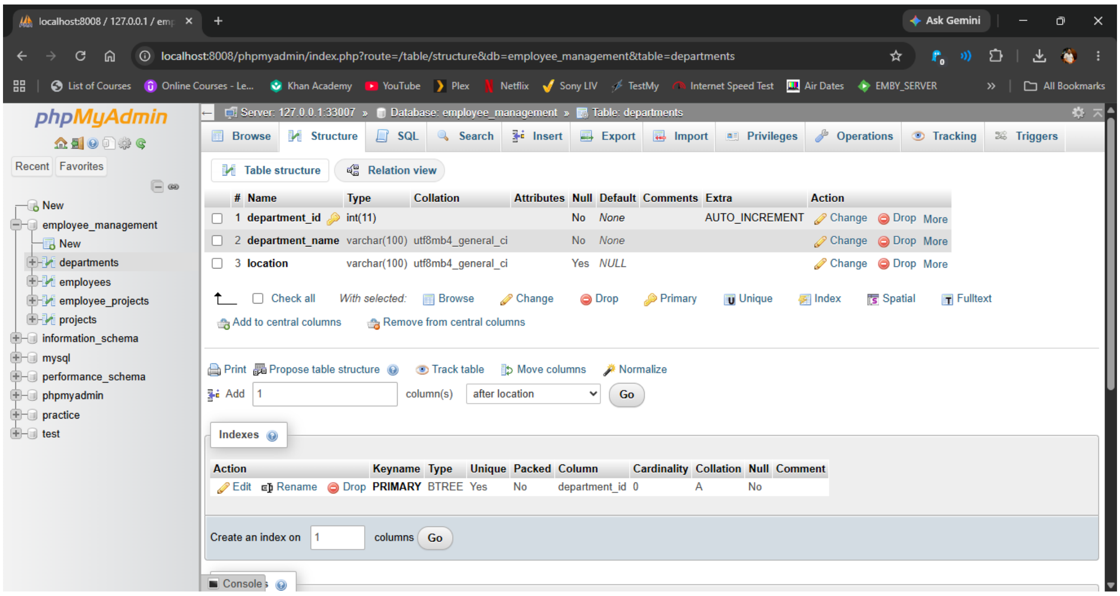

Figure 13: phpMyAdmin structure departments table.

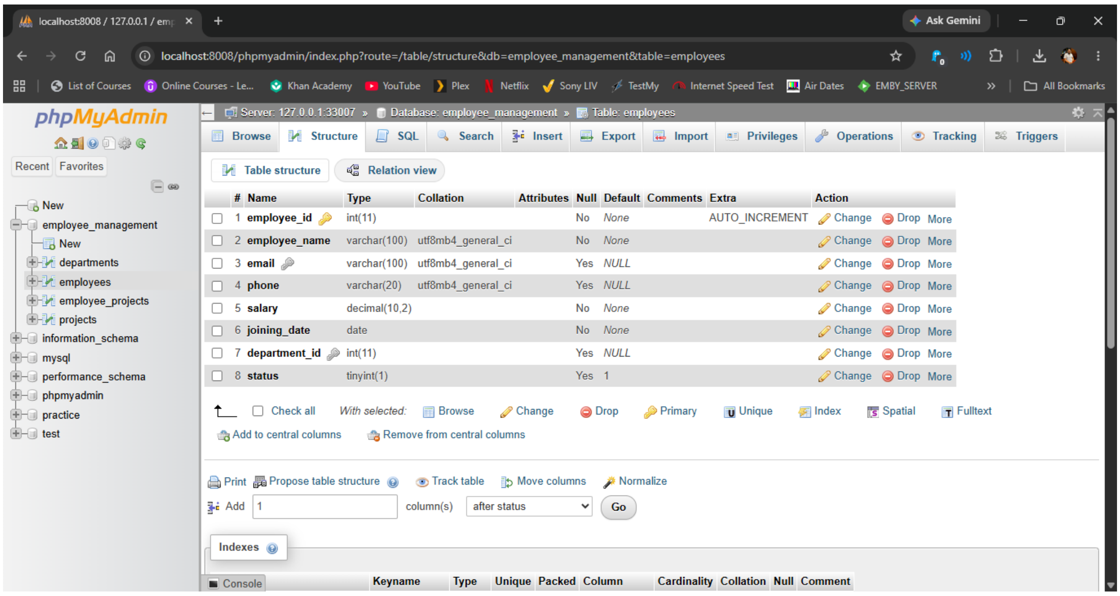

Figure 14: phpMyAdmin structure employees table.

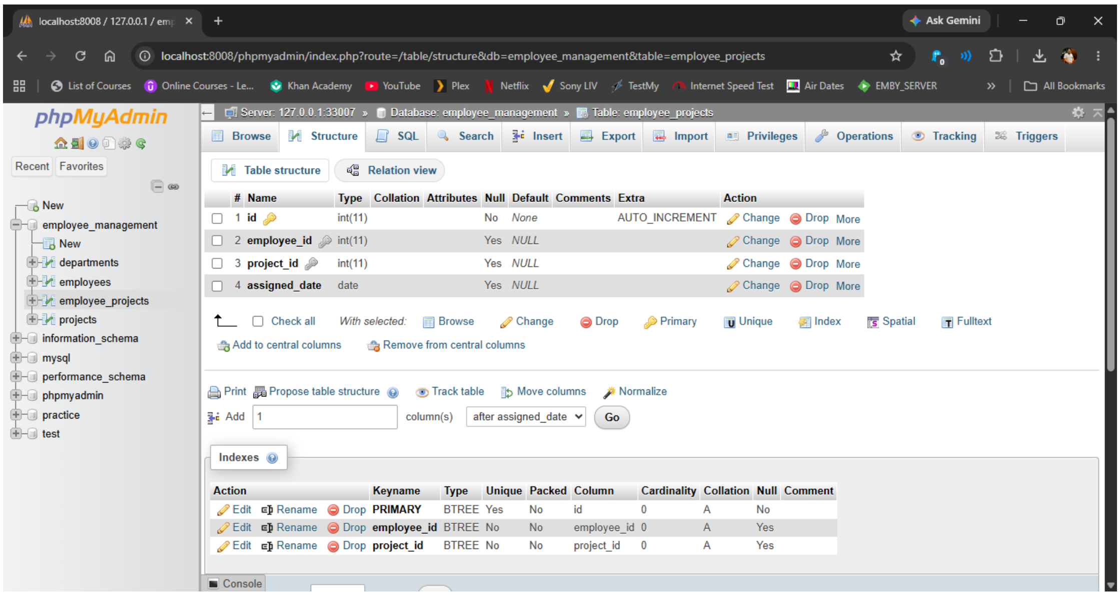

Figure 15: phpMyAdmin structure employee projects table.

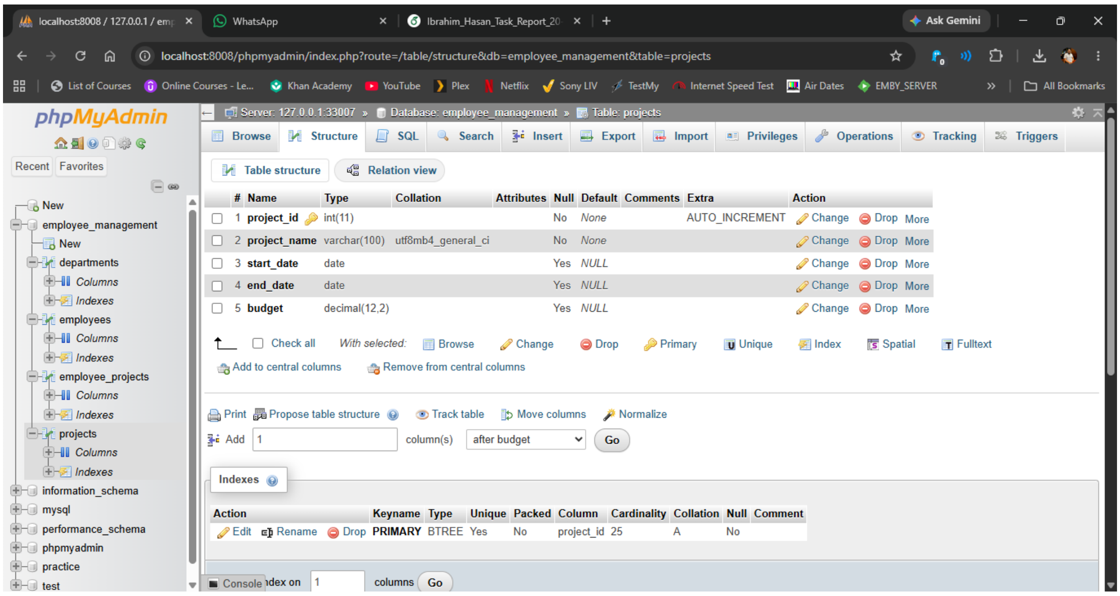

Figure 16: phpMyAdmin structure projects table.

Project Outcome

The `employee_management` database was successfully designed, implemented, and validated. All four tables were created with correct schema definitions, populated with 25 rows each (100 total records), and verified through SQL queries without error. Foreign key relationships between `departments`, `employees`, `projects`, and `employee_projects` are fully operational and consistent with the intended relational design. The environment is confirmed stable and ready for further development.